Dyad Practice Solutions

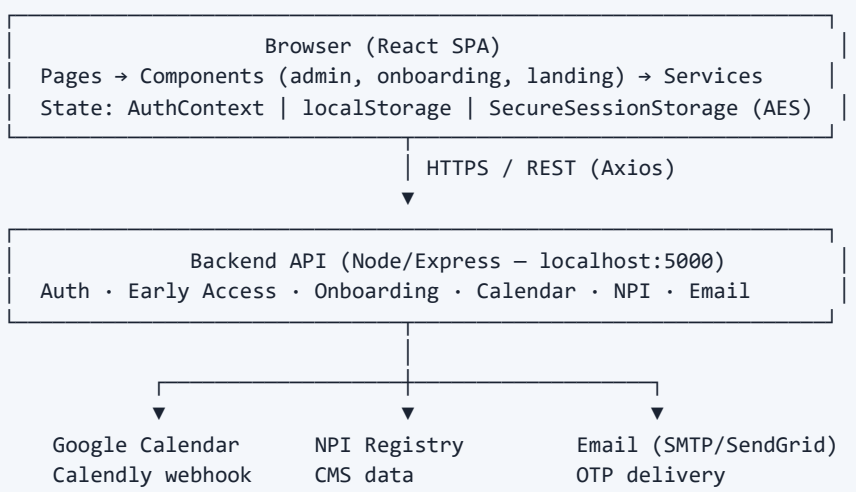
Application Architecture Report

Project: healthcare-app (newui)
Report Date: June 12, 2026
Type: Frontend SPA with REST backend dependency
Stack: React 19 · TypeScript · Vite 8 · React Router 7 · Axios

1. Executive Summary

> Dyad Practice Solutions is a healthcare-focused single-page application combining marketing, early-access intake, multi-step client onboarding, and admin operations. The frontend (React + Vite) communicates with a separate Node/Express-style backend via REST. Three parallel user journeys exist: public marketing, authenticated admin operations, and OTP-gated client onboarding.

2. High-Level Architecture



3. Technology Stack

Layer	Technology	Purpose
UI	React 19.2	Component-based UI
Language	TypeScript 5.9	Type safety
Build	Vite 8	Dev server, bundling
Routing	React Router 7	Client-side routing
HTTP	Axios 1.13	API with interceptors
Forms	React Hook Form + Yup	Validation
Styling	Tailwind + custom CSS	Hybrid styling
Security	crypto-js, reCAPTCHA	Token obfuscation, bot protection

4. Project Structure

```
src/
├── App.tsx              Root router, security init
├── pages/               DyadOnboarding, AdminEarlyAccess, EarlyAccess, Login…
├── components/
│   ├── admin/           Outreach, Active Clients, Schedule modals
│   ├── onboarding/      Steps 3–6 (agreements, due diligence, MSA, banking)
│   ├── landing/         Marketing sections
│   └── auth/            LoginWithCodeForm
├── contexts/            AuthContext (JWT)
├── services/            13 API/domain services
├── hooks/               useOnboardingSessionGuard
└── utils/               security, sessionStorage, validation
```

5. Routing & Access Control

Route	Component	Protection
/	DyadLanding	Public
/early-access	EarlyAccess	Public
/client-onboarding-process	DyadOnboarding	OnboardingProtectedRoute (OTP)
/dyad-onboarding-access	DyadOnboarding	Public
/admin-early-access	AdminEarlyAccess	Admin localStorage token
/admin	Admin	JWT ProtectedRoute (admin)
/user/*	ComingSoon	JWT ProtectedRoute (user)

Three Parallel Auth Systems

- **AuthContext (JWT)** — Login/register/OTP; encrypted sessionStorage; used for /admin and /user
- **Admin Early Access** — POST /admin/login; plain localStorage (adminAccessToken)
- **Onboarding Client** — Email OTP; dyad_onboarding_client_session (24h TTL)

6. Feature Domains

6.1 Marketing & Landing

- Hero, about, services, contact forms with reCAPTCHA
- Calendly embed for scheduling

6.2 Early Access

- Public intake with NPI validation and email OTP
- Admin review: submissions, beta cohort, invite emails

6.3 Client Onboarding (6 Steps)

Step	Name	Features
1	Overview	NPI lookup, practice type
2	Schedule Intro Call	Google Calendar picker, email confirmation
3	Sign Agreements	NDA, BAA, e-signature
4	Due Diligence	Document uploads
5	Commercial Alignment	MSA, exhibits, fee schedule
6	Bank & Payment	W-9, Zoho Pay (UI mock)

- **Persistence:** localStorage (dyad_onboarding_v1) + POST /onboarding/step/:n

6.4 Admin Operations

- **Outreach Schedule** — Schedule calls, send emails, calendar integration
- **Active Clients** — Onboarding records, messaging
- **Onboarding Pipeline, Reports, Settings** — Stubs (nav disabled)

7. Services Layer

Service	Responsibility
api.ts	Axios instance, JWT interceptors, token refresh
onboardingCalendarService.ts	Call scheduling, availability slots
onboardingStorageService.ts	Step-bucketed localStorage schema
onboardingHydrationService.ts	Server → local state merge
npiRegistryService.ts	NPI lookup, taxonomy mapping
emailService.ts + schedule services	Email templates, backend relay
onboardingClientAuthService.ts	OTP login, client session
onboardingSecurityService.ts	Idle timeout, session cleanup

8. External Integrations

Integration	Usage	Status
Google Calendar	Onboarding & admin scheduling	**Live via backend**
Calendly	ContactUs, ScheduleCall	**Embed**
reCAPTCHA	Contact form	**Live**
NPI Registry	Early access, onboarding	**Via backend**
Email (SMTP/SendGrid)	OTP, confirmations	**Backend relay**
Zoho Pay	Step 6 banking	**UI mock only**

9. Security Features

Implemented

Feature	Location	Description
JWT + refresh	AuthContext, api.ts	Bearer auth, 401 retry
Encrypted session	sessionStorage.ts	AES obfuscation (crypto-js)
Session timeout	SessionTimeout	30 min session, 25 min warning
Idle timeout	useOnboardingSessionGuard	9 min warning, 10 min logout
OTP verification	Login, register, early access	Email OTP flows
Route guards	ProtectedRoute, OnboardingProtectedRoute	Role & session checks
CSRF utilities	security.ts	Client-generated CSRF
Input sanitization	security.ts	InputSanitizer
Security monitor	securityMonitor.ts	Suspicious activity, concurrent sessions
reCAPTCHA	ContactForm	Bot protection

Security Gaps

Issue	Risk	Recommendation
CSP/headers disabled	XSS, clickjacking	Enable via server headers
Hardcoded encryption key	Token exposure	HttpOnly cookies
Admin plain localStorage	XSS token theft	Secure server sessions
Refresh logout disabled	Stale sessions	Re-enable on 401
Demo mode	Auth bypass	Disable in production
Three auth systems	Inconsistent security	Consolidate

10. Scalability Analysis

Strengths

- SPA + REST — frontend scales via CDN/static host
- Service layer — clear API separation
- Step-based onboarding — incremental persistence
- Backend offloads NPI, calendar, email

Limitations

Area	Limitation	Impact
DyadOnboarding.tsx	~3,300 lines monolith	Hard to maintain, slow load
localStorage	No cross-device sync	Poor multi-device UX
No state library	Context + localStorage only	Complexity at scale
No API caching	Re-fetch on navigation	Unnecessary load
Admin lists	No pagination	Performance at scale

Recommendations

- Split onboarding into lazy-loaded step routes
- Add React Query/SWR for caching
- Server-side sessions (Redis) for onboarding
- Pagination for admin lists
- CDN for static assets

11. Pros and Cons

Pros

#	Advantage
1	Modern stack — React 19, TypeScript, Vite
2	Clear domain separation — landing, early access, onboarding, admin
3	Structured 6-step onboarding with progress persistence
4	Multiple auth flows for different user types
5	Security utilities — timeout, idle guard, OTP, RBAC
6	Integrations — Calendar, NPI, email, Calendly
7	Offline resilience — localStorage draft saves
8	Form validation — React Hook Form + Yup
9	Admin tooling — outreach, active clients
10	Dev-friendly — Vite HMR, APIproxy

Cons

#	Disadvantage
1	Monolithic onboarding page (~3,300 lines)
2	Three separate auth systems
3	Client-side token storage (XSS risk)
4	Security headers disabled
5	Mock integrations (Zoho Pay, demo mode)
6	No automated test suite
7	Mixed styling (Tailwind + large CSS files)
8	API URL inconsistency across services
9	Disabled admin features (pipeline, reports)
10	Tight backend coupling, no OpenAPI client

12. Conclusion

> **Overall assessment:** Solid foundation for MVP/pilot deployment. The application has a well-defined service layer, multi-step onboarding, and admin tooling. For production scale, prioritize: (1) security hardening — HttpOnly cookies, CSP, consolidated auth; (2) refactor onboarding monolith; (3) complete Zoho Pay integration; (4) add tests and API caching.

13. Deployment

- **Frontend:** npm run build → static host / CDN
- **Backend:** Node server (e.g. Railway) at VITE_API_URL
- **Env:** VITE_API_URL, VITE_RECAPTCHA_SITE_KEY, calendar/email vars

---